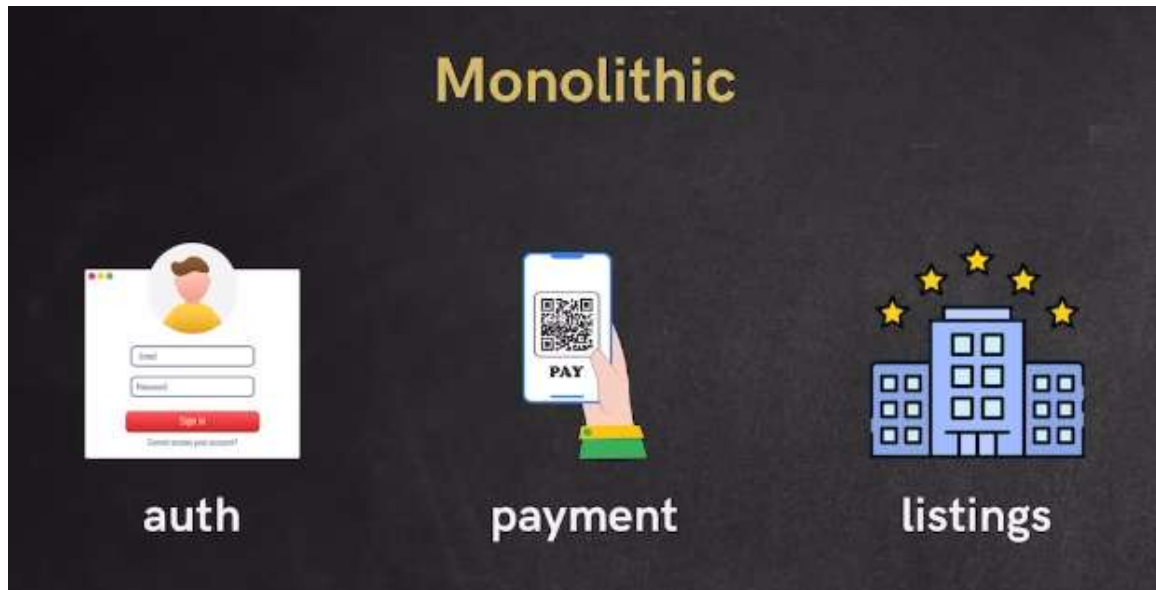


Monolithic vs Microservices

What is Monolithic Architecture?

Monolithic Architecture is a software design approach where the entire application is built, developed, and deployed as a single unit.



All modules such as authentication, payments, orders, notifications, and products exist inside one codebase and are packaged together into a single deployable artifact such as:

- Single **.jar** (Java) / Single **.war** / Single **.exe** / Single **Docker container**

Develop as a single unit, deploy as a single unit — single CI/CD pipeline



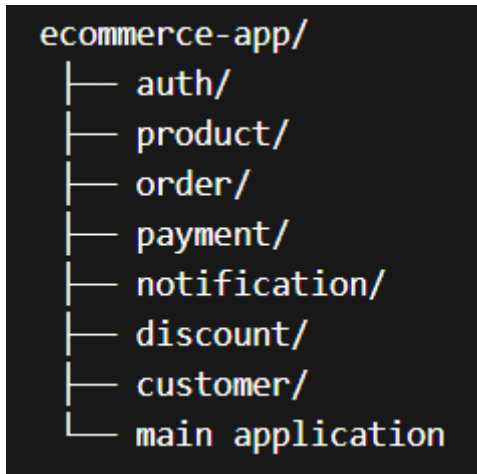
Monolithic Architecture → Ecommerce Application

Consider an Ecommerce application like Amazon with the following modules:

- Customer Module
- Authentication Module
- Product Module
- Order Module
- Payment Module
- Notification Module
- Discount Module

In Monolithic architecture:

All these modules exist inside:



Everything runs as:

ecommerce.jar

Deployment:

Server → Run ecommerce.jar → Entire application runs

CI/CD:

Code change → Build entire app → Deploy entire app

How Development Works in Monolithic

Single repository → Single codebase → Single build pipeline

If developer updates:

Auth module password encryption logic

Entire application must be:

Rebuilt → Retested → Redeployed

Even though only one module changed.

How Deployment Works in Monolithic

Deployment unit:

Single Application Artifact

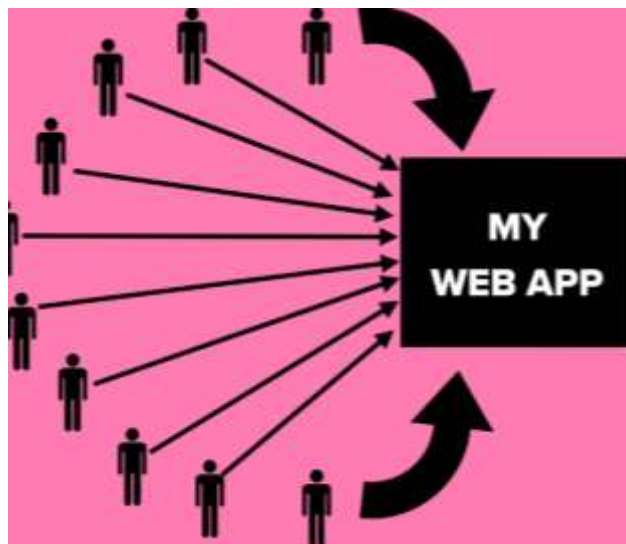
Example:

docker run ecommerce-app

All services run together.

Scaling in Monolithic Architecture

Scaling means handling more users or traffic.

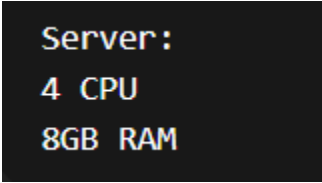


Vertical Scaling

Increasing server capacity.

Example:

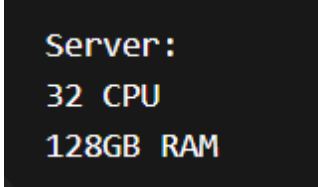
Before:



```
graph TD; S[Server: 4 CPU 8GB RAM];
```

Server:
4 CPU
8GB RAM

After scaling:



```
graph TD; S[Server: 32 CPU 128GB RAM];
```

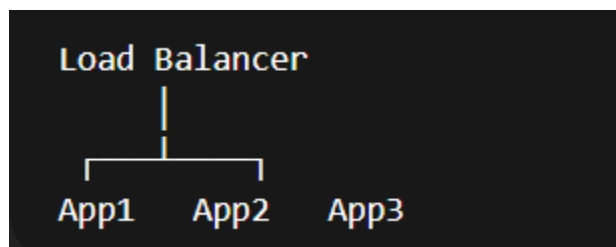
Server:
32 CPU
128GB RAM

Problem:

- Expensive
- Hardware limit exists

Horizontal Scaling

Running multiple instances of same application.



Each instance contains:

1. Auth
2. Payment
3. Order
4. Product

Even if only auth needs scaling → Waste of resources.

Problems in Monolithic Architecture

1. Redeployment Problem

Example:

➔ Fix bug in Auth module.

Still must deploy entire app.

Process:

```
Change auth code  
Build entire app  
Test entire app  
Deploy entire app
```

Time consuming & Risky.

2. Scaling Limitations

Example:

During signup campaign:

➔ Auth receives heavy traffic.

But cannot scale only auth.

Must scale entire application.

Wasteful.

3. Dependency Conflicts

Example:

Auth module needs:

```
JWT Library v1.2
```

Product module uses:

JWT Library v1.0

Since single application:

Must use same version.

Causes conflicts.

4. Tightly Coupled System

Modules depend heavily on each other.

Example:

Payment module directly calls order functions.

Changing order logic may break payment.

Hard to modify safely.

5. Slow Development in Large Teams

Ex : 1000 developers working on one codebase.

1. Merge conflicts.
2. Integration issues.
3. Long build times.

6. Technology Lock-in

Entire application uses same tech stack.

Example:

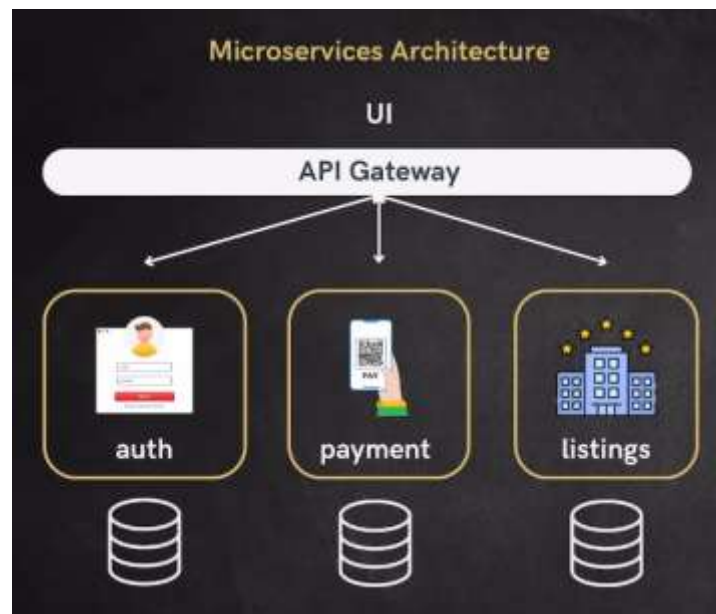
- If built in Java:
- All modules must use Java.
- Cannot use Python for ML recommendation.

What is Microservices Architecture?

Microservices Architecture is a design approach where an application is divided into **small, independent services**, each responsible for specific business functionality.

Each service:

- Has independent codebase
- Has independent deployment
- Has independent database
- Communicates via APIs or messages



Ecommerce Application with Microservices

Instead of one application:

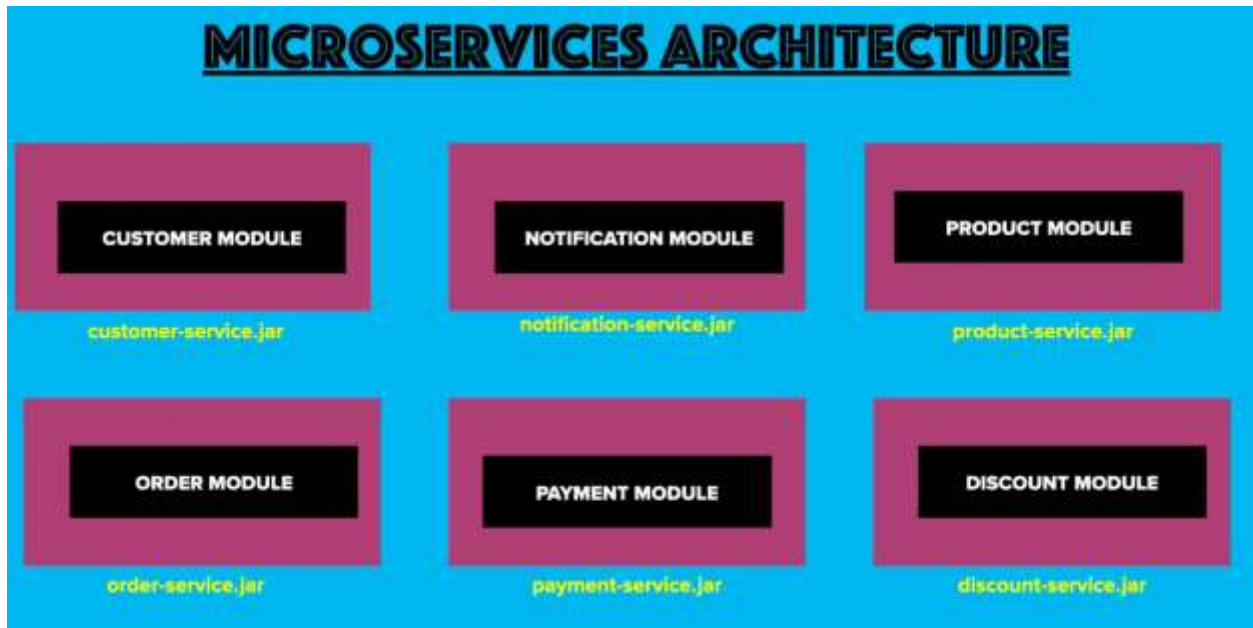
We create separate services:

1. Auth Service
2. Product Service
3. Order Service
4. Payment Service
5. Notification Service
6. Customer Service
7. Discount Service

Each runs independently.

Example deployment:

1. auth-service.jar
2. payment-service.jar
3. order-service.jar



Each Service Has

Independent:

1. Codebase
2. Repository
3. Team
4. Deployment
5. CI/CD pipeline

Example:

1. Auth team deploys auth service.
2. Payment team deploys payment service.

Independently.

How Ecommerce Flow Works in Microservices

Example: Customer buys product.

Step 1

Customer → Order Service

Step 2

Order Service → Payment Service

Step 3

Payment success → Notification Service

Step 4

Notification Service sends email

Each handled by different services.

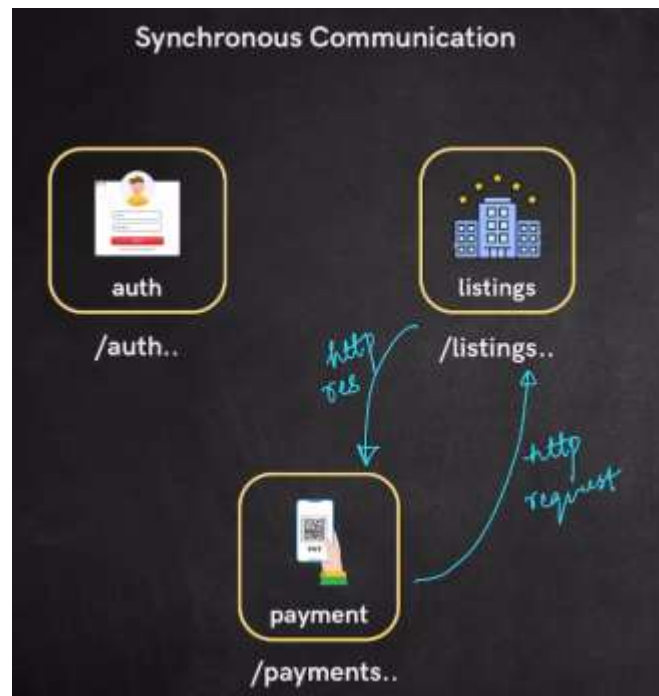
How Microservices Communicate

Method 1: Synchronous Communication (REST API)

Service calls another service directly.

Example:

- Payment Service calls listings service.
- POST /listings/process
- Request → Response



Each microservice will have individual endpoint

1. Authentication → /auth
2. Payment → /payment
3. Listings → /listings

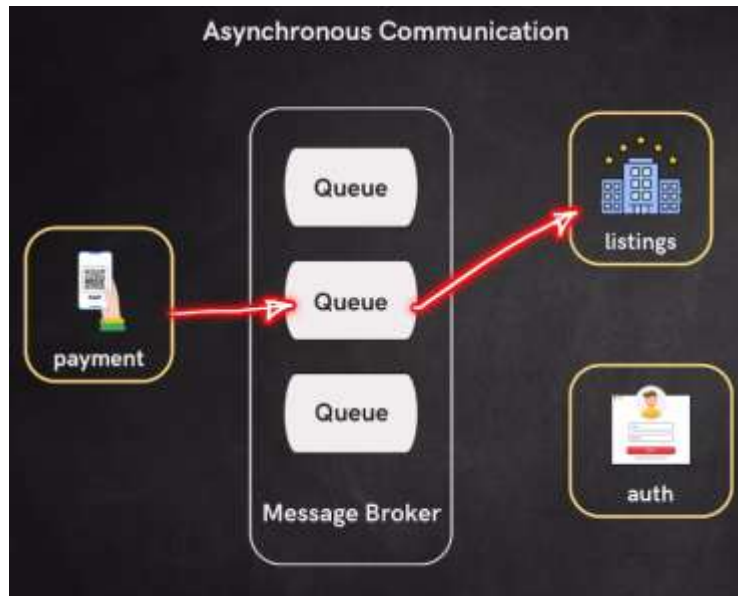


Method 2: Asynchronous Communication (Message Broker)

Services communicate using message broker.

Message brokers:

- ➔ Kafka
- ➔ RabbitMQ
- ➔ ActiveMQ



Example:

1. Payment Service sends message:
2. Payment done.
3. Kafka stores message.
4. Listings Service consumes message.
5. Processes listings.
6. No direct dependency.

Method 3: Service Mesh (Istio)

Used in Kubernetes.

Handles:

- Service discovery
- Security
- Routing

Automatically.

Scaling in Microservices

Example:

➔ During Big Billion Day sale.

Heavy load on:

➔ Order Service

Scale only Order Service, Not entire system.

Real Life Examples of Microservices

Taxi Booking App (Uber)

Services:

1. User Service
2. Driver Service
3. Location Service
4. Payment Service
5. Notification Service

Booking flow:

1. User Service → Booking Service
2. Booking Service → Driver Service
3. Driver accepts
4. Payment Service processes payment

Money Transfer App

Services:

1. Account Service
2. Payment Service
3. Fraud Service
4. Notification Service

Transfer flow:

1. User initiates transfer
2. Payment Service processes
3. Notification sent

Amazon Ecommerce Example

Services:

1. User Service
2. Catalog Service
3. Search Service
4. Cart Service
5. Order Service
6. Payment Service
7. Shipping Service

Benefits of Microservices

1. Independent Deployment

Example:

- Auth service version:
 - v1 → v2
- Deploy only auth service.
- No need to deploy payment service.

2. Technology Flexibility

1. Search Service → Java
2. Payment Service → Python
3. Sellers → .Net

Each service chooses best technology.



3. Independent Scaling

Example:

High login traffic.

Scale only auth service.

Not entire application.



4. Faster Development

Teams work independently.

Parallel development possible.

5. Fault Isolation

If notification fails:

Payment still works.

System does not crash entirely.

Disadvantages of Microservices

1. Complex Architecture

- Multiple services.
- Hard to manage.

2. Infrastructure Cost

Need:

- Servers
- Containers
- Networking
- Higher cost.

3. Debugging Difficulty

1. Request flows across many services.
2. Harder debugging.

4. Requires Skilled Teams

Not suitable for small startups.

Migration from Monolithic to Microservices

1: Does each file become microservice?

NO, Microservices are based on BUSINESS FUNCTION, not files.

Example:

Auth service contains:

1. Login
2. Register
3. JWT
4. User validation

All grouped into one service.

Not separate services for each file.

2: How many microservices should we create?

There is NO fixed number.

Depends on:

1. Business requirements
2. Team structure
3. System complexity

Example Ecommerce microservices:

- Auth Service
- Product Service
- Order Service
- Payment Service
- Notification Service

Large companies may have 500+ services.

Small companies may have 5 services.

3: How to restructure codebase?

Monolithic:

```
ecommerce/  
  auth/  
  payment/  
  order/
```

Microservices:

```
auth-service/  
payment-service/  
order-service/
```

Separate repositories.

Separate deployments.

4: How services communicate?

In Monolithic:

Communication via function calls:

```
processPayment()
```

In Microservices:

Communication via:

HTTP REST API

Example:

```
POST /payment/process
```

When to Use Monolithic

- Small applications
- Small teams
- Simple systems

Example:

Startup MVP

When to Use Microservices

- Large applications
- Large teams
- High traffic systems

Example:

1. Amazon
2. Netflix
3. Uber

Monolithic architecture builds and deploys the entire application as a single unit, making it simpler but difficult to scale and maintain at large scale.

Microservices architecture divides the application into independent services, enabling independent deployment, scaling, and technology flexibility, but introduces infrastructure and operational complexity.

Modern large-scale systems like Amazon, Netflix, and Uber use microservices to handle millions of users efficiently.

Feature	Monolithic	Microservices
Deployment	Single unit	Independent
Scaling	Entire app	Individual service
Development	Single codebase	Multiple codebases
Flexibility	Low	High
Complexity	Low	High
Best for	Small teams	Large systems

Important Design Patterns in Microservices Architecture

Microservices architecture introduces flexibility and scalability, but it also introduces complexity. To manage this complexity, several **design patterns** are used. These patterns help define clear service boundaries, enable communication, manage deployment, and ensure scalability and reliability.

1. Domain Driven Design (DDD) Pattern

Domain Driven Design (DDD) is a design approach used to **define clear boundaries for each microservice based on business functionality (domain)**.

Each microservice represents a specific business capability and contains only the logic related to that domain.

Each microservice should focus on one business responsibility only.

Why DDD is Important in Microservices

In microservices, improper separation can cause confusion and tight coupling.

DDD ensures:

- Clear ownership of services
- Independent development
- Easier scalability
- Easier maintenance

Example rule:

1. Email notification logic should NOT exist inside the Payment service.
2. Order processing logic should NOT exist inside the Authentication service.

Each service must handle only its own responsibility.

Ecommerce Example using DDD

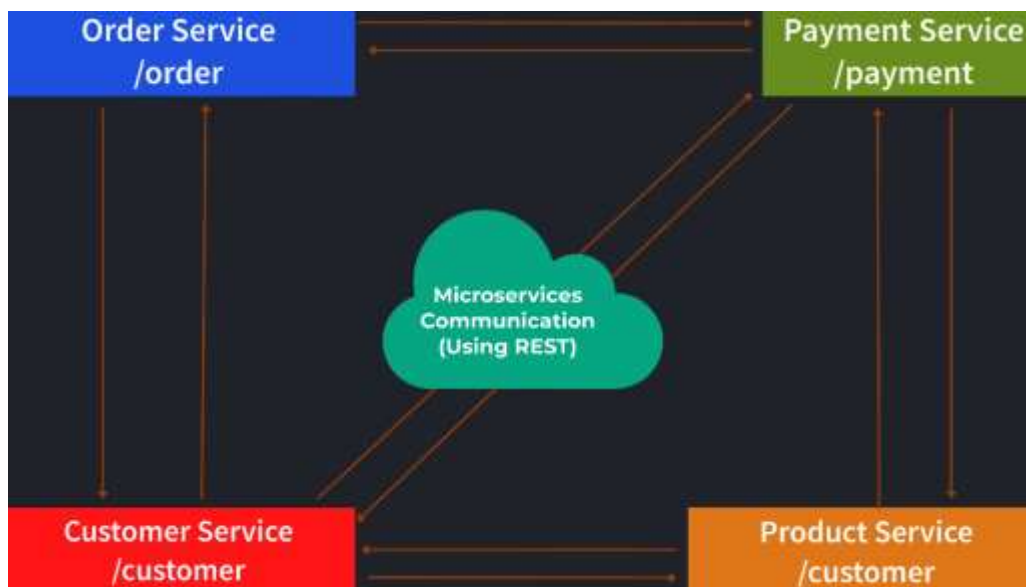
Microservices division:

1. Authentication Service → login, register, JWT validation
2. Product Service → product listing, search
3. Order Service → create order, track order
4. Payment Service → process payments, refunds
5. Notification Service → email, SMS notifications

Example flow when placing an order:

1. User clicks "Place Order"
2. Order Service creates order
3. Order Service calls Payment Service
4. Payment Service processes payment
5. Notification Service sends confirmation

Each service performs only its own responsibility.



Loan Application Example using DDD

Consider a banking loan application.

Instead of building one large system, we divide it into domain-specific services:

1. Auto Loan Service → handles car loan applications
2. Home Loan Service → handles home loan processing
3. Recovery Service → handles loan recovery
4. Agent Service → manages loan agents

Each service has its own:

- Codebase
- Database
- Deployment



Key Principle of DDD

Define clear **Bounded Context**

Bounded context means → Each service owns its domain completely.

Payment service handles → Payment processing Refunds

It does NOT handle:

1. Order creation
2. Email sending

Domain Knowledge Requirement

Microservices developers must understand the business domain clearly.

Example:

If building Payment Service, developer must understand:

1. Payment flow
2. Payment status
3. Refund logic

Without domain knowledge, incorrect service boundaries may be created.

2. Service Discovery and Registration Pattern

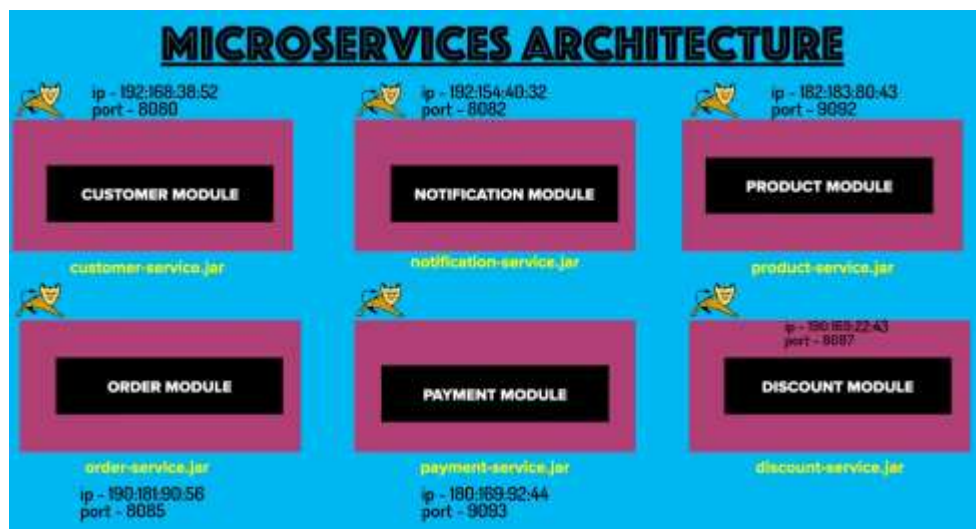
Problem

In microservices, services run on:

- different Servers
- different IP addresses
- different Ports

Example:

1. Order Service → 10.0.0.2:8080
2. Payment Service → 10.0.0.5:9090
3. Product Service → 10.0.0.7:8085



If Order Service wants to call Payment Service, how does it know:

- ➔ IP address?
- ➔ Port number?

Hardcoding is not possible because services scale dynamically.

Solution: Service Discovery

Service Discovery is a system that stores and provides the location of all services.

It acts like a **phone directory of microservices**.

Examples:

1. Eureka
2. Consul
3. Kubernetes Service Discovery

How Service Registration Works

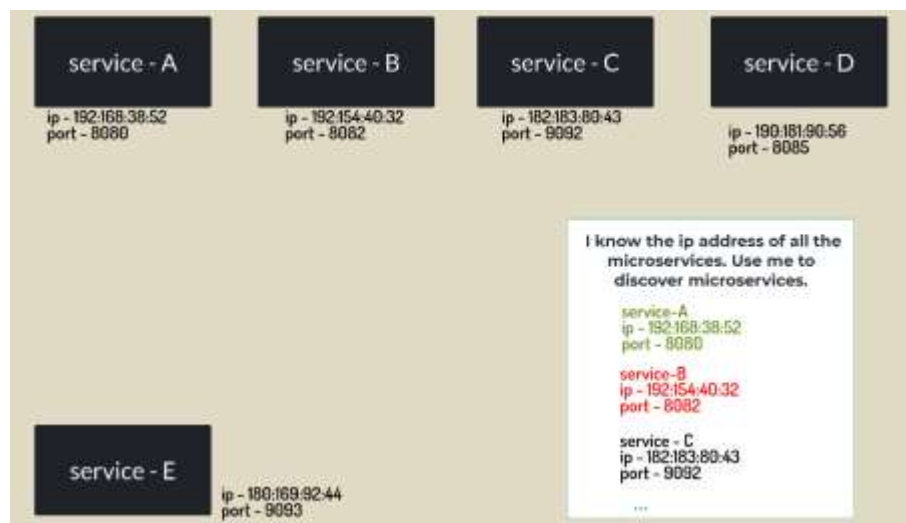
When a microservice starts, it registers with Discovery Service.

It provides:

1. Service Name
2. IP Address
3. Port

Example:

Payment Service registers → 10.0.0.5:9090



How Service Discovery Works

Example:

Order Service wants to call Payment Service.

Step 1

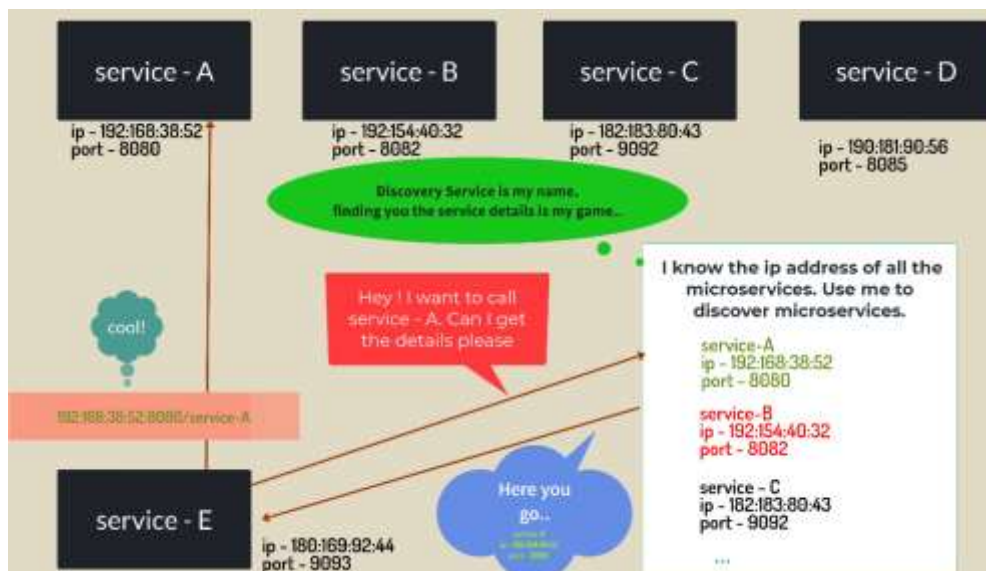
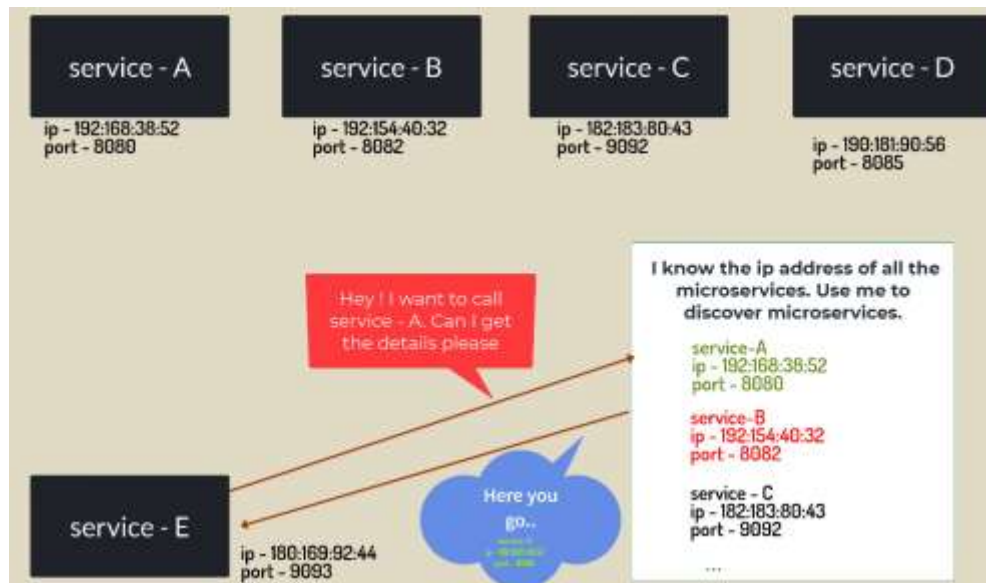
Order Service asks Discovery Service → **"Where is payment-service?"**

Step 2

Discovery Service returns → 10.0.0.5:9090

Step 3

Order Service calls Payment Service.



3. Microservice Communication using Message Brokers

Previously, services communicated directly using REST APIs.

But direct communication creates tight dependency.

Solution:

Message Broker

Message broker acts as a **middleman between services**.

Instead of calling directly, services send messages to brokers.

Broker forwards message to destination service.

Common Message Brokers:

1. Apache Kafka
2. RabbitMQ
3. ActiveMQ
4. Apache Artemis

4. Message Broker Pattern

Message Broker is a system that enables **asynchronous communication between microservices using messages**.

Instead of direct communication:

Service → Message Broker → Destination Service

Example Ecommerce Flow

1. Order Service creates order.
(Instead of calling Payment Service directly)
2. Order Service sends message to Kafka → "Order Created"
3. Kafka stores message.
4. Payment Service reads message.
5. Processes payment.

Producer and Consumer Concept

Producer → Service sending message.

Ex: Order Service

Consumer → Service receiving message.

Ex: Payment Service

5. Event Streaming Pattern

What is an Event?

An event represents something that happened in the system.

Examples in Ecommerce:

1. Order created
2. Payment completed
3. Item shipped
4. Out for delivery
5. Refund initiated

Each of these is an event.



**E commerce
App**

Application Events

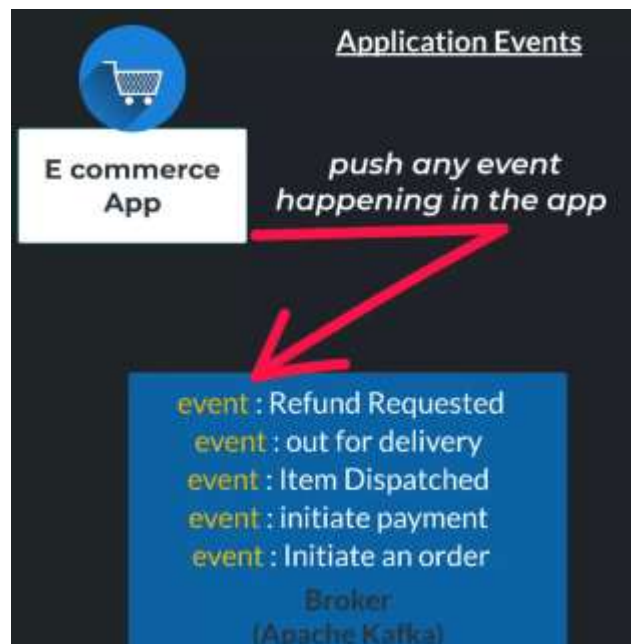
event : Initiate an order
event : initiate payment
event : Item Dispatched
event : out for delivery
event : Refund Requested

How Event Streaming Works

Services publish events to Message Broker.

Example:

1. Order Service publishes event:
2. Order Shipped
3. Kafka receives event
4. Notification Service consumes event
5. Customer receives notification



Benefits:

1. Loose coupling
2. High scalability
3. Real-time processing

6. Database Design Pattern in Microservices

Monolithic Database Design

Single database used by entire application.

Example:

ecommerce_db

Tables:

1. Users
2. Orders
3. Payments
4. products

All modules access same database.

Problem:

- Tight coupling
- Scaling difficulty

Microservices Database Design

Each microservice has its own database.

Example:

1. Auth Service → auth_db
2. Order Service → order_db
3. Payment Service → payment_db

Benefits:

- Independent scaling
- Data isolation
- Improved reliability

Example:

➔ Payment Service failure does not affect Order database.

7. Microservice Deployment using Docker and Kubernetes

Each microservice is packaged into a container using Docker.

Example:

auth-service → Docker container

payment-service → Docker container

Docker ensures consistent environment.

Kubernetes manages:

Deployment

Scaling

Load balancing

Health monitoring

Example:

If Payment Service receives heavy traffic, Kubernetes automatically creates more instances.

8. Load Balancer Pattern

Problem

If one service instance receives too much traffic, it becomes overloaded.

Example:

Order Service running on:

→ Server 1

→ Server 2

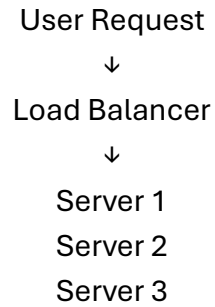
→ Server 3

Users must be distributed across servers.

Solution: Load Balancer

Load Balancer distributes traffic evenly across service instances.

Architecture:



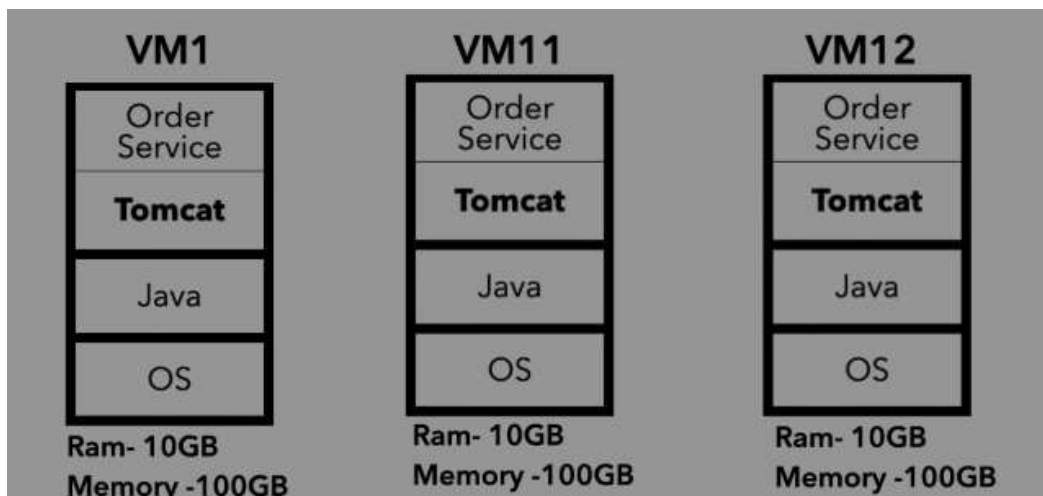
Example Ecommerce Application:

During sale:

Millions of users place orders.

Load balancer distributes requests across multiple Order Service instances.

Prevents system crash.



Benefits

- Improved performance
- High availability
- Fault tolerance

If one server fails, load balancer redirects traffic to others.

Summary

Key microservice patterns include:

1. Domain Driven Design → defines service boundaries
2. Service Discovery → helps services find each other
3. Message Brokers → enable asynchronous communication
4. Event Streaming → enables real-time event processing
5. Database per Service → ensures isolation
6. Docker & Kubernetes → manage deployment and scaling
7. Load Balancer → distributes traffic efficiently

These patterns enable microservices to be scalable, flexible, and reliable in large-scale production systems.